

React Escape Hatches: Hinglish Explanation

React normally chahta hai ki **UI aur data flow declarative** rahe. Matlab aap React ko batate ho ki UI kaisi honi chahiye, aur React DOM ko automatically update kar deta hai.

Lekin kuch situations mein component ko React ke bahar maujood systems se interact karna padta hai, jaise:

- Input field ko programmatically focus karna
- Video ko play ya pause karna
- Browser DOM element access karna
- Server connection establish karna
- Timer ya event listener manage karna
- Third-party library control karna

In situations ke liye React kuch features deta hai jinhe **Escape Hatches** kaha gaya hai.

Important idea: Escape hatches exceptional situations ke liye hain. Application ka normal data flow mostly props, state aur event handlers par based hona chahiye.

1. Values ko bina re-render ke remember karna: useRef

Kabhi component ko koi information remember karni hoti hai, lekin us information ke change hone par UI update nahi karni hoti.

Aise case mein useRef use hota hai:

```
1 const ref = useRef(0);
```

Ref ki current value:

```
1 ref.current
```

se access aur modify ki jaati hai.

Example

```
1 import { useRef } from 'react';
2
3 export default function Counter() {
4   const ref = useRef(0);
5
6   function handleClick() {
7     ref.current = ref.current + 1;
8     alert('You clicked ' + ref.current + ' times!');
```

```
9    }
10
11    return (
12      <button onClick={handleClick}>
13        Click me!
14      </button>
15    );
16  }
```

Button click hone par ref.current increase hota hai, lekin component re-render nahi hota.

State aur Ref mein difference

```
1  setState(newValue);
```

State update:

- Value remember karta hai
- Component ko re-render karta hai
- UI mein dikhne wale data ke liye useful hai

```
1  ref.current = newValue;
```

Ref update:

- Value remember karta hai
- Component ko re-render nahi karta
- Internal ya non-visual information ke liye useful hai

Refs kahan useful hain?

Refs mein aap store kar sakte ho:

- Timeout ID
- Interval ID
- DOM element
- Previous value
- Third-party library instance
- Aisi mutable information jo rendering ko affect nahi karti

Golden rule

Agar value screen par dikhni chahiye aur change hone par UI update honi chahiye, to **state use karo**.

Agar value sirf internally remember karni hai, to **ref use kar sakte ho**.

Ref component ki ek “secret pocket” jaisa hai jise React rendering ke purpose se track nahi karta.

2. DOM ko refs se access karna

React normally DOM ko khud manage karta hai. Isliye manually DOM manipulation frequently nahi karni chahiye.

Lekin kuch browser operations ke liye actual DOM node chahiye hota hai, jaise:

- Input focus karna
- Element tak scroll karna
- Element ka size measure karna
- Position check karna
- Video ya audio method call karna

Iske liye DOM element par ref attach kiya jaata hai.

```
1  import { useRef } from 'react';
2
3  export default function Form() {
4    const inputRef = useRef(null);
5
6    function handleClick() {
7      inputRef.current.focus();
8    }
9
10   return (
11     <>
12       <input ref={inputRef} />
13
14       <button onClick={handleClick}>
15         Focus the input
16       </button>
17     </>
18   );
19 }
```

Step-by-step flow

1. Ref create hua

```
1  const inputRef = useRef(null);
```

Initially inputRef.current ki value null hai.

2. Ref input ke saath connect hua

```
1 <input ref={inputRef} />
```

React DOM input element ko inputRef.current mein assign kar deta hai.

3. DOM method call hua

```
1 inputRef.current.focus();
```

Yahan focus() browser DOM API hai. React API nahi hai.

Important caution

Refs ka use karke manually har cheez modify mat karo:

```
1 inputRef.current.value = 'Hello';
```

Is tarah React ke controlled data flow aur actual DOM ke beech inconsistency ho sakti hai.

Refs ko mainly un operations ke liye use karo jo React declaratively handle nahi karta, jaise focus, scrolling aur measurements.

3. External systems ke saath synchronize karna: useEffect

useEffect ka central purpose hai:

Component ko React ke bahar ke kisi external system ke saath synchronize karna.

External system ho sakta hai:

- Browser API
- Network connection
- Chat server
- Video player
- Analytics system
- Event listener
- Timer
- Third-party component

Video example

```
1 function VideoPlayer({ src, isPlaying }) {  
2   const ref = useRef(null);  
3  
4   useEffect(() => {
```

```
5     if (isPlaying) {
6         ref.current.play();
7     } else {
8         ref.current.pause();
9     }
10 }, [isPlaying]);
11
12 return {src};
13 }
```

Yahan React state ya prop `isPlaying` ko browser ke actual video player ke saath synchronize kiya ja raha hai.

- `isPlaying === true` → video play
- `isPlaying === false` → video pause

Effect har render ke baad automatically nahi chalega. Dependency array mein `[isPlaying]` hai, isliye relevant synchronization `isPlaying` ke change hone par hota hai.

4. Event handler aur Effect mein difference

Event handlers aur Effects ka purpose different hota hai.

Event handler

Event handler kisi particular user interaction ke response mein chalta hai.

```
1 function handleClick() {
2     setIsPlaying(!isPlaying);
3 }
```

Ye tab chalta hai jab user button click karta hai.

Effect

Effect rendering ke baad external system ko synchronize karta hai.

```
1 useEffect(() => {
2     ref.current.play();
3 }, [isPlaying]);
```

Ye `isPlaying` ki latest value ke according video ko synchronize karta hai.

Simple way se:

- **Event handler:** User ne kya action kiya?
- **Effect:** Latest render ke according external system ko kya karna chahiye?

5. Effect cleanup

Kai Effects external resource setup karte hain. Jab component remove ho ya Effect dobara chale, old resource ko properly close karna zaroori hota hai.

Example:

```
1  useEffect(() => {
2    const connection = createConnection();
3
4    connection.connect();
5
6    return () => {
7      connection.disconnect();
8    };
9  }, []);
```

Effect ke andar return ki gayi function ko **cleanup function** kehte hain.

Is example mein

Effect setup:

```
1  connection.connect();
```

Cleanup:

```
1  connection.disconnect();
```

Cleanup ka use in situations mein hota hai:

- Server disconnect karna
- Event listener remove karna
- Timer cancel karna
- Subscription unsubscribe karna
- Third-party instance destroy karna

Example pattern:

```
1  useEffect(() => {
2    window.addEventListener('resize', handleResize);
3
4    return () => {
5      window.removeEventListener('resize', handleResize);
6    };
7  }, []);
```

Agar cleanup nahi karoge, to duplicate listeners, stale connections aur memory-related problems aa sakti hain.

6. Development mein Effect do baar kyon dikh sakta hai?

Development environment mein React Effect ko ek additional setup-cleanup cycle se test kar sakta hai.

Isliye console mein kuch aisa dikh sakta hai:

```
1 Connecting...
2 Disconnected.
3 Connecting...
```

Iska purpose ye check karna hai ki Effect ki cleanup properly likhi gayi hai ya nahi.

Aapko duplicate log ko hide karne ke liye Effect ko hack nahi karna chahiye. Better approach hai:

- Setup correct ho
 - Cleanup setup ko properly reverse kare
 - Effect repeated setup-cleanup handle kar sake
-

7. Har jagah Effect ki zarurat nahi hoti

Effects React ke normal data flow se bahar jane ke liye hote hain. Agar koi external system involved nahi hai, to generally Effect ki zarurat nahi honi chahiye.

Do common cases:

1. Rendering ke liye data calculate karna
2. User event handle karna

Unnecessary Effect

```
1 function Form() {
2   const [firstName, setFirstName] = useState('Taylor');
3   const [lastName, setLastName] = useState('Swift');
4
5   const [fullName, setFullName] = useState('');
6 }
```

```
7   useEffect(() => {
8     setFullName(firstName + ' ' + lastName);
9   }, [firstName, lastName]);
10 }
```

Yahan fullName independently change nahi ho raha. Woh firstName aur lastName se derive ho raha hai.

Is wajah se separate state aur Effect redundant hain.

Better approach

```
1  function Form() {
2    const [firstName, setFirstName] = useState('Taylor');
3    const [lastName, setLastName] = useState('Swift');
4
5    const fullName = firstName + ' ' + lastName;
6  }
```

Is approach ke benefits:

- Extra render nahi
- State duplication nahi
- Synchronization bug ka risk kam
- Code simple
- Data ka single source of truth

Useful decision

Apne aap se poochho:

Kya main kisi external system ke saath synchronize kar raha hoon?

Agar answer **no** hai, to Effect use karne se pehle check karo ki calculation render ke andar ya action event handler mein ho sakti hai ya nahi.

8. Effect ka lifecycle component se different hai

Component lifecycle ke common concepts:

- Mount
- Update
- Unmount

Effect ko doosre perspective se samjho:

1. Synchronization start karna
2. Synchronization stop karna

Ye cycle component ki lifetime mein multiple times ho sakti hai.

```
1  useEffect(() => {  
2    const connection = createConnection(serverUrl, roomId);  
3  
4    connection.connect();  
5  
6    return () => {  
7      connection.disconnect();  
8    };  
9  }, [roomId]);
```

Suppose initial roomId hai:

```
1  general
```

Effect general room se connect karega.

Agar roomId change hokar ban jaaye:

```
1  travel
```

React ka conceptual sequence hoga:

1. General room ki old connection disconnect
2. Travel room ki new connection establish

Isliye Effect ko sirf “component mount par chalne wala code” samajhna incomplete hai.

Better mental model:

Effect kisi particular set of reactive values ke according synchronization process represent karta hai.

9. Reactive values kya hoti hain?

Component ke andar kuch values re-render ke saath change ho sakti hain:

- Props
- State
- Component body ke andar calculate ki gayi values
- Props aur state se derived variables

Example:

```
1 function ChatRoom({ roomId }) {  
2   const [message, setMessage] = useState('');  
3 }
```

Yahan:

- roomId reactive hai
- message reactive hai

Agar Effect roomId read karta hai:

```
1 useEffect(() => {  
2   const connection = createConnection(roomId);  
3   connection.connect();  
4  
5   return () => connection.disconnect();  
6 }, [roomId]);
```

to roomId dependency list mein hona chahiye.

React ka linter missing dependencies detect karne mein help karta hai.

10. Events aur Effects ko separate karna

Event handlers tab re-run hote hain jab corresponding interaction dobara hota hai.

Effects tab re-synchronize hote hain jab unki reactive dependencies change hoti hain.

Problem tab hoti hai jab Effect ke andar aisi value read ki jaaye jiske change hone par synchronization restart nahi honi chahiye.

Problem example

```
1 useEffect(() => {  
2   const connection = createConnection(serverUrl, roomId);  
3  
4   connection.on('connected', () => {  
5     showNotification('Connected!', theme);  
6   });  
7  
8   connection.connect();  
9  
10  return () => connection.disconnect();  
11 }, [roomId, theme]);
```

Effect roomId aur theme dono read kar raha hai.

Isliye:

- roomId change → reconnect
- theme change → reconnect

Lekin theme change hone par chat connection dobara establish karna unnecessary hai. Theme sirf notification appearance ke liye chahiye.

Effect Event approach

```
1  const onConnected = useEffectEvent(() => {
2    showNotification('Connected!', theme);
3  });
4
5  useEffect(() => {
6    const connection = createConnection(serverUrl, roomId);
7
8    connection.on('connected', () => {
9      onConnected();
10    });
11
12    connection.connect();
13
14    return () => connection.disconnect();
15  }, [roomId]);
```

Ab:

- roomId change → Effect re-run aur connection update
- theme change → Chat reconnect nahi
- Connected callback latest theme use kar sakta hai

Core distinction

Effect ka reactive part:

```
1  createConnection(serverUrl, roomId);
```

Non-reactive event-like part:

```
1  showNotification('Connected!', theme);
```

useEffectEvent aise logic ko Effect ke reactive synchronization se separate karta hai.

11. Effect dependencies ko blindly remove mat karo

React linter check karta hai ki Effect ke andar read ki gayi reactive values dependency array mein present hain ya nahi.

```
1  useEffect(() => {  
2    // Effect logic  
3  }, [dependency]);
```

Dependencies ko intentionally hide ya suppress karna correct solution nahi hota.

Problem: object har render par recreate hota hai

```
1  const options = {  
2    serverUrl,  
3    roomId  
4  };  
5  
6  useEffect(() => {  
7    const connection = createConnection(options);  
8    connection.connect();  
9  
10   return () => connection.disconnect();  
11  }, [options]);
```

JavaScript mein har render par ek naya object create hota hai:

```
1  { serverUrl, roomId }
```

Values same hone par bhi object reference naya hota hai.

Result:

- User input mein type karta hai
- State update hoti hai
- Component re-render hota hai
- Naya options object banta hai
- Effect ko dependency changed lagti hai
- Chat unnecessary reconnect hota hai

Wrong solution

```
1  useEffect(() => {  
2    const connection = createConnection(options);  
3  }, []);
```

Sirf dependency remove kar dena wrong hai, kyunki Effect ab latest roomId ke saath synchronize nahi rahega.

Correct solution

Object creation Effect ke andar move karo:

```
1  useEffect(() => {
2    const options = {
3      serverUrl,
4      roomId
5    };
6
7    const connection = createConnection(options);
8    connection.connect();
9
10   return () => connection.disconnect();
11 }, [roomId]);
```

Ab Effect sirf actual reactive primitive roomId par depend karta hai.

Important principle

Dependency array manually select ki hui wishlist nahi hai.

```
1  [roomId]
```

Ye Effect ke code ka description hai: Effect ka reactive synchronization roomId par depend karta hai.

Dependency list ko forcefully change karne ke bajaye surrounding code change karo, taaki unnecessary dependency naturally remove ho jaaye.

12. Custom Hooks se logic reuse karna

React built-in Hooks provide karta hai:

- useState
- useEffect
- useRef
- useContext

Lekin application mein repeated specialized behavior ho sakta hai, jaise:

- Online status track karna
- Chat room connect karna
- Pointer position track karna
- Window size track karna

- Delayed value maintain karna

Aise reusable behavior ko **Custom Hook** mein extract kiya ja sakta hai.

Custom Hook function ka naam conventionally use se start hota hai.

Pointer position Hook

```
1  import { useState, useEffect } from 'react';
2
3  export function usePointerPosition() {
4    const [position, setPosition] = useState({
5      x: 0,
6      y: 0
7    });
8
9    useEffect(() => {
10     function handleMove(e) {
11       setPosition({
12         x: e.clientX,
13         y: e.clientY
14       });
15     }
16
17     window.addEventListener('pointermove', handleMove);
18
19     return () => {
20       window.removeEventListener('pointermove', handleMove);
21     };
22   }, []);
23
24   return position;
25 }
```

Component mein:

```
1  const position = usePointerPosition();
```

Ab component ko event listener setup aur cleanup ki details nahi sambhalni padti.

13. Custom Hooks logic share karte hain, state nahi

Agar do components ek custom Hook call karte hain:

```
1  const position1 = usePointerPosition();
2  const position2 = usePointerPosition();
```

to dono calls ka state independent hota hai.

Custom Hooks ka purpose:

- Stateful logic share karna
- Implementation details hide karna
- Components ko readable banana
- Repeated Effect logic centralize karna

Custom Hook call karne se automatically ek globally shared state nahi ban jaata.

14. Hooks ko compose karna

Ek custom Hook doosre custom Hook ke output ko input ke roop mein use kar sakta hai.

Example:

```
1  const pos1 = usePointerPosition();
2  const pos2 = useDelayedValue(pos1, 100);
3  const pos3 = useDelayedValue(pos2, 200);
```

Yahan:

- pos1 current pointer position
- pos2 delayed position
- pos3 aur zyada delayed position

Is tarah Custom Hooks ko small reusable building blocks ki tarah compose kiya ja sakta hai.

15. Delayed value example aur cleanup consideration

Provided example mein:

```
1  export function useDelayedValue(value, delay) {
2    const [delayedValue, setDelayedValue] = useState(value);
3
4    useEffect(() => {
5      setTimeout(() => {
6        setDelayedValue(value);
7      }, delay);
8    });
9  }
```

```
8     }, [value, delay]);  
9  
10    return delayedValue;  
11  }
```

Jab value ya delay change hota hai, Effect naya timeout schedule karta hai.

Is example ka main purpose delayed values ko compose karke cursor trail dikhana hai. Broader lesson ye hai ki custom Hooks Effects ko internally use karke reusable behavior expose kar sakte hain.

Complete mental model

useState

UI ko affect karne wali reactive information:

```
1  const [count, setCount] = useState(0);
```

State update se re-render hota hai.

useRef

Render ko affect na karne wali mutable information:

```
1  const timerRef = useRef(null);
```

ref.current change karne se re-render nahi hota.

Event handler

User interaction ke response mein code:

```
1  function handleClick() {  
2    setCount(count + 1);  
3  }
```

useEffect

External system ko rendered state ke saath synchronize karna:

```
1  useEffect(() => {  
2    connection.connect();  
3  
4    return () => connection.disconnect();  
5  }, []);
```

Effect Event

Effect ke andar event-like non-reactive logic ko reactive synchronization se separate karna:

```
1  const onConnected = useEffectEvent(() => {  
2    showNotification(theme);  
3  });
```

Custom Hook

Reusable stateful behavior package karna:

```
1  function usePointerPosition() {  
2    // Reusable Hook logic  
3  }
```

Practical decision guide

Kya value change hone par UI update honi chahiye?

Use:

```
1  useState
```

Kya value remember karni hai, lekin render nahi karwana?

Use:

```
1  useRef
```

Kya actual DOM element access karna hai?

Use:

```
1  useRef
```

aur element par ref attach karo:

```
1  <input ref={inputRef} />
```

Kya user interaction handle karna hai?

Use:

```
1  onClick  
2  onChange  
3  onSubmit
```

jaise event handlers.

Kya React ke bahar kisi system ko synchronize karna hai?

Use:

```
1 useEffect
```

Kya props/state se sirf data calculate karna hai?

Effect mat use karo. Rendering ke time calculate karo:

```
1 const fullName = firstName + ' ' + lastName;
```

Kya repeated Hook-based logic multiple components mein hai?

Custom Hook banao:

```
1 function useSomething() {  
2   // reusable logic  
3 }
```

Common mistakes

1. Ref value ko UI mein show karna

```
1 ref.current++;
```

Agar updated value UI mein dikhani hai, ref correct choice nahi hai, kyunki ref update re-render nahi karta.

Use state instead.

2. Derived value ko Effect aur state mein rakhna

```
1 useEffect(() => {  
2   setTotal(price * quantity);  
3 }, [price, quantity]);
```

Agar total directly calculate ho sakta hai:

```
1 const total = price * quantity;
```

to Effect unnecessary hai.

3. Cleanup bhool jana

```
1 useEffect(() => {
```

```
2   window.addEventListener('resize', handleResize);
3 }, []);
```

Better:

```
1  useEffect(() => {
2    window.addEventListener('resize', handleResize);
3
4    return () => {
5      window.removeEventListener('resize', handleResize);
6    };
7  }, []);
```

4. Dependency warning suppress karna

```
1  useEffect(() => {
2    connect(roomId);
3  }, []);
```

Agar Effect roomId read karta hai, to empty dependency array stale synchronization create kar sakti hai.

Correct approach:

```
1  useEffect(() => {
2    connect(roomId);
3  }, [roomId]);
```

Ya code restructure karo agar dependency unnecessary behavior cause kar rahi hai.

5. Objects aur functions ko unnecessary dependencies banana

Render ke andar banaye gaye object ka reference har render mein different ho sakta hai:

```
1  const options = { roomId };
```

Isliye Effect unnecessarily re-run ho sakta hai. Relevant object creation ko Effect ke andar move karna useful ho sakta hai.

Final summary

React ke **Escape Hatches** आपको React ke controlled rendering model ke bahar ke systems ke saath interact karne dete hain.

Is chapter ke main lessons:

1. **useRef value remember karta hai, lekin re-render trigger nahi karta.**
2. **DOM element ko focus, scroll ya measure karne ke liye ref use hota hai.**
3. **useEffect external systems ke saath synchronization ke liye hai.**
4. **Effect ke setup ko cleanup function se reverse karna chahiye.**
5. **Derived data calculate karne aur user events handle karne ke liye generally Effect nahi chahiye.**
6. **Effect ka lifecycle start synchronization aur stop synchronization ke around hota hai.**
7. **Dependency array Effect ke reactive code ko describe karti hai.**
8. **Dependencies ko blindly remove nahi karna chahiye. Code restructure karna chahiye.**
9. **Effect Events reactive synchronization aur non-reactive event logic ko separate karte hain.**
10. **Custom Hooks reusable stateful logic ko components ke beech share karte hain.**

Ek line mein:

State UI ke liye, ref non-rendering memory ke liye, event handler interaction ke liye, Effect external synchronization ke liye, aur Custom Hook reusable behavior ke liye hota hai.